HYPERION RESEARCH

Free CLI Agentic Workflow

A bounded five-role coordination architecture for bring-your-own-key strategy authoring, durable review, and fresh-run repair

IMPLEMENTATION STATUS

This paper describes the implemented Hyperion Free Strategy Workflow baseline. The five named agents are fixed serial role invocations inside one coordinator. The current gauntlet verifies process integrity only; it does not compile, execute, backtest, or certify generated strategy code.

Document	System design and operator assurance paper
Version	1.0 - 8 August 2026
Workflow schema	hyperion-free-strategy-workflow/v1
Policy	free-strategy-authoring-inert/v1
Prompt catalog SHA-256	bb79da228ed798981334970908f4cdb8739edb1e728f67b21156adbe101cdb0b
Audience	Project review, security review, Terminal integration, and research submission

Core claim. Reliability comes from narrow authority, immutable evidence, strict schemas, external deterministic checks, exact-hash approvals, and bounded retry - not from asking a language model to declare its own work correct.

Executive overview

The Hyperion Free Strategy Workflow lets a user supply a strategy brief, approved reference sources, and a model provider they control. Hyperion then runs five fixed roles in order: Planner, Evidence Analyst, Critic, Synthesizer, and Risk Judge. Each role receives the immutable objective, untrusted supplied sources, and only the earlier outputs that have already passed structural validation. The workflow grants the model no browser, filesystem, shell, compiler, package, broker, order, or execution tool.

The result is a hash-bound inert C# strategy artifact for human review. A deterministic eight-check process gauntlet verifies the durable event chain, approvals, role artifacts, decision, artifact lineage, authority flags, and release state. If revision is needed, Hyperion creates a new lineage-bound run. No source bytes or approvals are silently mutated or inherited.

WHAT IS IMPLEMENTED

Serial five-role orchestration; strict JSON and citation validation; durable SQLite event history; content-addressed artifacts; bounded budgets and retries; pinned BYOK provider profiles; eight process-integrity checks; exact-hash start and release approvals; and at most two fresh repair generations.

WHAT IS DEFERRED

Terminal-owned restricted compilation, deterministic functional tests, property and metamorphic tests, sandbox execution, and final registration authority. Diagnostics from those future gates may seed a separately approved repair run.

WHAT IS NOT GUARANTEED

A passing process gauntlet does not prove compilation, correctness, safety, profitability, robustness, regulatory suitability, or live-trading fitness. No probabilistic agent loop can honestly promise 100 percent semantic correctness.

How to read the paper

Part	Question answered
Architecture and roles	What calls what, what each role sees, and what each role may produce.
State and gauntlet	How approvals, hashes, durable artifacts, and deterministic process checks work.
Repair and budgets	How feedback creates fresh runs and where every loop stops.
BYOK and threats	How provider routing, secrets, network policy, and failure recovery are bounded.
Terminal boundary	Which stronger validation gates are deliberately outside this implementation.

1. Research basis and architecture decision

The design draws useful mechanisms from established agent and code-generation research while rejecting open-ended loops that weaken cost, authority, or auditability. ReAct interleaves reasoning and actions [1]; Reflexion and Self-Refine use feedback across attempts [2, 3]; language-agent tree search explores branches [4]; debate introduces competing arguments [5]; CRITIC and self-debugging ground revision in external observations [6, 7]; AlphaCodium and CodeT emphasize test-aware generation [8, 9]; and EvalPlus shows that shallow test suites can overstate correctness [10].

Hyperion adopts a narrower rule: model critique is advisory, while deterministic external evidence controls acceptance. The current free workflow implements process evidence and a diagnostics bridge. Compilation and behavioral evidence belong to the consuming Terminal, not to the model or the current CLI.

Research pattern	Useful idea	Hyperion decision
ReAct [1]	Reasoning linked to observations and actions.	No model tools. Hyperion sequences calls and validations.
Reflexion [2]	Feedback can inform a later attempt.	Exact parent artifacts and approved diagnostics enter a fresh run; no free-form persistent memory.
Self-Refine [3]	Generation, critique, and revision.	Critic and Risk Judge advise review but cannot certify correctness.
Tree search [4]	Branch, score, and backtrack.	Not used: branching cost and model-valued rollouts are incompatible with predictable free workflow bounds.
Debate [5]	Disagreement may expose assumptions.	Not represented as debate: roles are serial, may share one model, and do not vote.
CRITIC / self-debug [6, 7]	External diagnostics ground repair.	Diagnostics bridge implemented; compiler and execution tools deferred to Terminal.
AlphaCodium / CodeT [8, 9]	Test-oriented flow improves code selection.	Future Terminal gates rerun on exact replacement bytes; generated tests never override hard failures.
EvalPlus [10]	Expanded tests reveal false passes.	Use multiple independent gates and retain an explicit non-guarantee boundary.
Deep research [12, 13]	Plan, search, pivot, parallelize, and synthesize sources.	Not used: this workflow has no search or subagents and accepts only user-supplied sources.

DESIGN DECISION

One deterministic coordinator runs five fixed role prompts, validates every output, stores exact artifacts, and stops at explicit terminal states. A new repair generation is allowed only through an approved request and exact lineage. Search, debate, hidden candidates, and autonomous tools are not part of this workflow.

2. End-to-end architecture

The CLI is an operator surface around a durable coordinator. Configuration selects a reviewed provider profile. The coordinator constructs canonical role prompts, calls the selected provider serially, validates responses, and appends hash-bound events and artifacts. The gauntlet is a read-only inspection over durable state and makes no provider request.

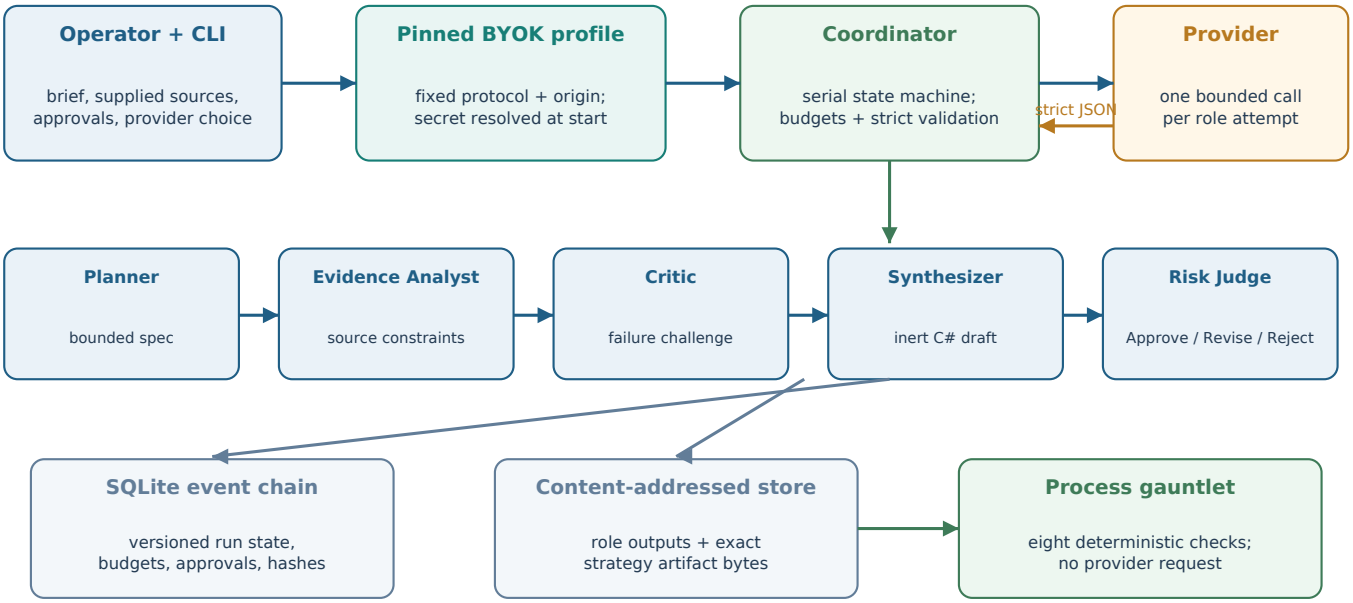


Figure 1. Implemented end-to-end path. Model calls are serial and tool-free. Durable stores and the process gauntlet remain under Hyperion control.

Component	Authority	Durable output
CLI	Collect configuration, sources, actor identity, exact hashes, and explicit approvals.	Reviewed config and command result.
Coordinator	Sequence roles, reserve and charge budgets, retry bounded failures, and apply the state machine.	Hash-chained events and snapshots.
Provider adapter	Perform one protocol-specific completion request within fixed origin and response limits.	Validated provider result only.
Role validator	Enforce schema, expected role, known citations, strategy file limits, and terminal decision rules.	Canonical role artifact.
Content-addressed store	Persist exact bytes under their SHA-256 identity.	Role and strategy artifacts.
Gauntlet	Recompute process facts from durable state without model discretion.	Versioned eight-check report.

3. How the five agents work

TERMINOLOGY

In the product experience, each named role is called an agent. In the implementation, it is a fixed role invocation inside one serial state machine - not a separately deployed autonomous process, identity, memory, or independently trained model.

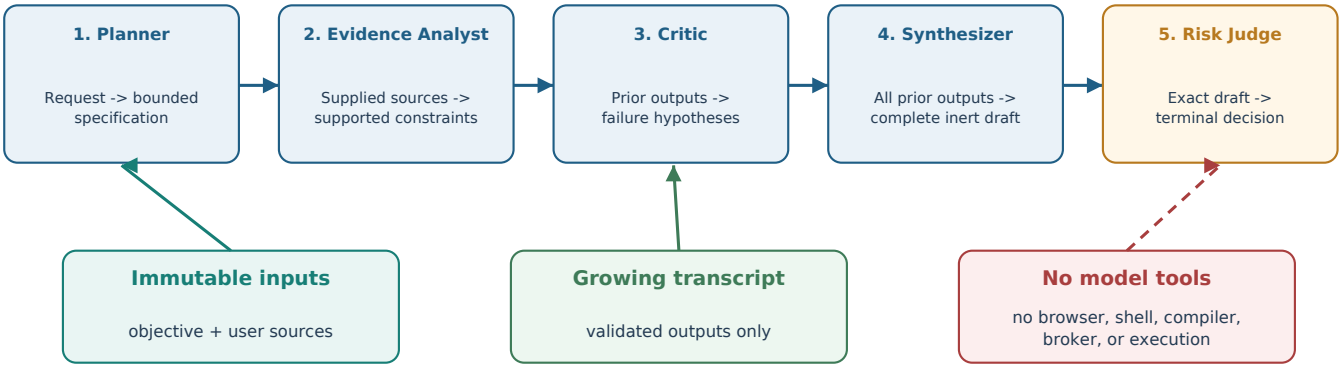


Figure 2. Role order and information growth. Every downstream role sees only immutable inputs and earlier validated outputs.

Order and role	Primary job	Permitted special output	Why it exists
1. Planner	Turn the brief into a bounded strategy specification and identify missing requirements.	No strategy source; decision must remain None.	Reduces ambiguity before source generation.
2. Evidence Analyst	Extract only supplied SDK, architecture, quantitative, and risk constraints.	Claims cite only known source IDs.	Separates evidence from model inference.
3. Critic	Challenge invented APIs, lookahead, unsafe order behavior, weak controls, and unverifiable assumptions.	Risks and recommendations; no acceptance authority.	Surfaces failure hypotheses before drafting.
4. Synthesizer	Produce one complete inert C# strategy using safe relative .cs paths.	The only role allowed to return a strategy draft.	Converts the validated context into exact reviewable bytes.
5. Risk Judge	Review the exact draft as inert text and choose Approve, Revise, or Reject.	The only role allowed to set a terminal decision.	Creates an explicit advisory disposition for the state machine.

Independence caveat

All five roles may be served by the same provider and model. Their errors can therefore be correlated. The role split improves task decomposition and review coverage, but it does not create statistical independence, formal verification, or a majority vote. This is why Risk Judge approval alone is never a semantic guarantee.

4. Prompt composition, output validation, and artifact identity

For role index i, Hyperion composes a role-specific system policy, the immutable objective, escaped source records, and validated outputs from roles 1 through $i - 1$. Source text is explicitly marked as untrusted data, so instructions embedded inside a source are not part of coordinator policy.

$$P_i = Compose(S_i, Objective, Sources, Y_{1:i-1}) \quad and \quad Y_i = Provider(P_i)$$

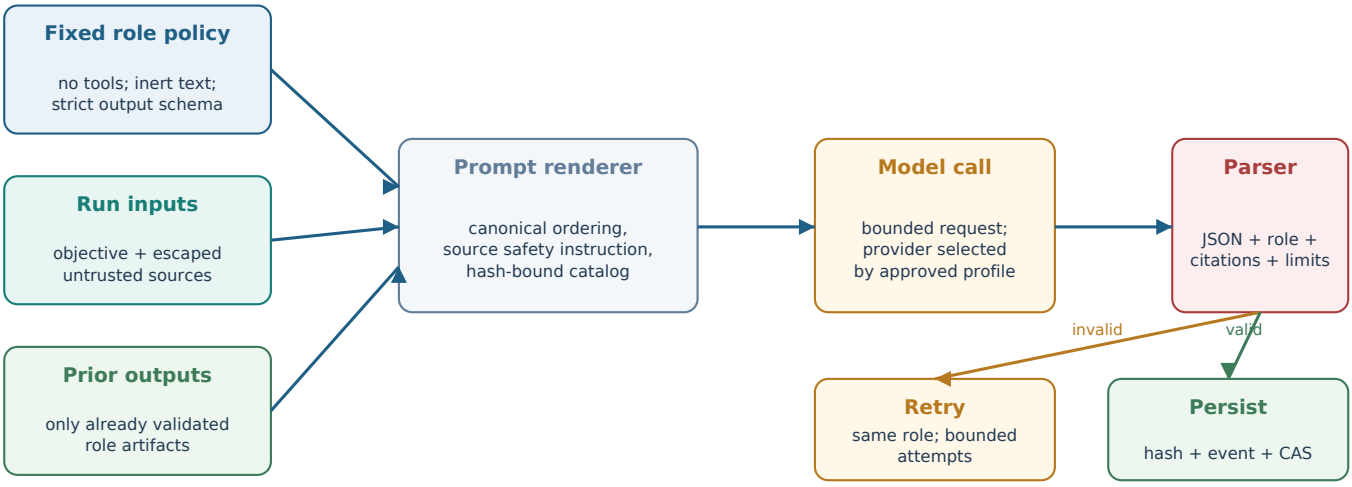


Figure 3. A role output reaches durable workflow state only after protocol, schema, role, citation, and size checks succeed.

Validation predicate

$$Valid_i = Schema_i(Y_i) \; AND \; RoleMatch_i \; AND \; Citations(Y_i) \; subset \; Sources \; AND \; Limits_i$$

The strict role schema rejects unknown fields. Citations must name supplied source IDs. Only the Synthesizer may include a strategy draft; only the Risk Judge may choose a terminal decision. A draft must contain one to sixteen non-empty C# files, safe relative paths, no more than 200,000 characters per file, and no more than 1,000,000 aggregate characters. These are structural controls, not C# parsing or correctness checks.

Content identity

$$h(x) = SHA\text{-}256(CanonicalBytes(x)) \quad and \quad ArtifactId = h(exact \; persisted \; artifact)$$

The run binds its policy, workflow version, prompt-catalog hash, provider descriptor, budget, specification, approvals, role outputs, and final artifact. The strategy artifact is reproducible from the exact Synthesizer output, Risk Judge output, evidence IDs, and run identity. Hashing does not prove that content is correct; it proves which content was reviewed and whether it changed.

5. Durable state machine and exact approvals

Creation stores a specification without contacting a provider. Starting requires an actor and the exact specification SHA-256. The coordinator records one start approval, runs or resumes serial roles, and transitions according to the Risk Judge decision or a bounded failure. Human release is a separate approval bound to both the specification and the final artifact hash.

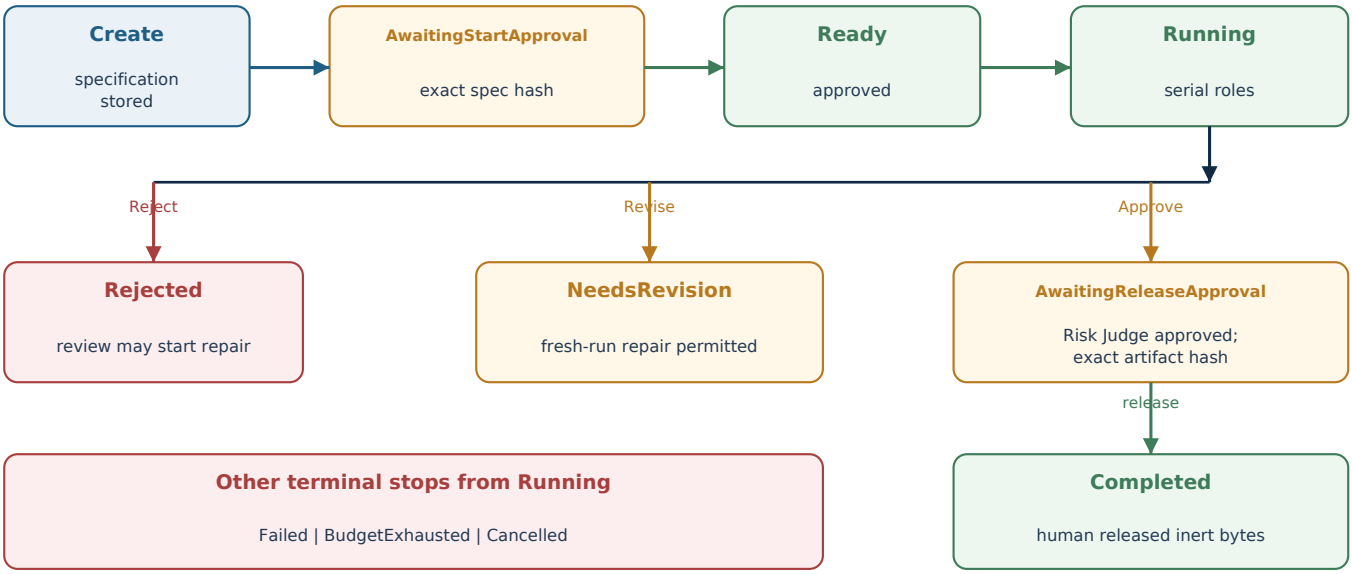


Figure 4. Principal states and outcomes. Explicit cancellation is terminal. A repair is a separate run and therefore does not appear as an in-place transition.

$$StartApproval = (RunId, h(Specification), Actor, Time) \quad ReleaseApproval = (RunId, h(Specification), h(Artifact), Actor, Time)$$

Resume and crash behavior

- Repeating the exact start command resumes an approved Ready or Running run; it does not create another run or approval.
- A still-live invocation is not duplicated. After request timeout plus 30 seconds, stale work is marked interrupted and its reserved budget is conservatively charged before bounded retry.
- If a host dies after the provider accepted a request but before durable completion, external dispatch and billing may remain ambiguous. Hyperion does not claim exactly-once provider execution without a provider-backed idempotency receipt.
- Explicit operator cancellation transitions to Cancelled and does not resume.

6. The implemented eight-check process gauntlet

The gauntlet is a deterministic inspection of durable workflow integrity. It performs no model call and does not compile or execute source. Integrity failures fail closed before a successful report can be returned.

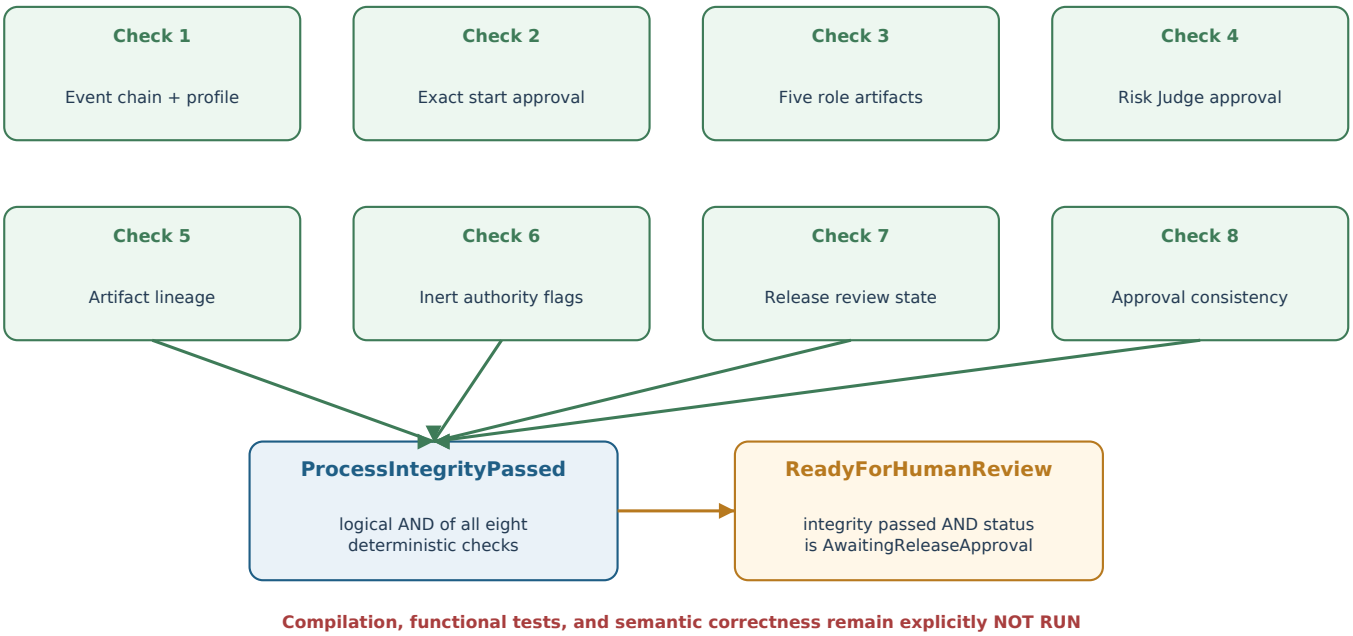


Figure 5. The report passes only when every process-integrity predicate passes. Human review readiness also requires the exact release-review state.

$$ProcessIntegrityPassed(run) = C_1 \ AND \ C_2 \ AND \ \ldots \ AND \ C_8$$

ID	Implemented assertion
1. verified-event-chain-and-profile	Durable events load and the run uses the exact free-strategy policy and workflow profile.
2. exact-start-approval	Exactly one start approval binds the current specification hash and no artifact.
3. complete-role-artifacts	Exactly one validated artifact exists for each of the five roles.
4. risk-judge-approval	The exact Risk Judge artifact says Approve.
5. strategy-artifact-lineage	The final artifact hash and contents reproduce from the exact run, Synthesizer, Risk Judge, and evidence IDs.
6. inert-authority-boundary	Human release and Terminal compile consent are required; installEligible is false; delivery state is inert.
7. release-review-state	The run is awaiting exact release review or is already completed.
8. release-approval-consistency	Awaiting review has no release approval; Completed has exactly one approval bound to spec and artifact.

A malformed event chain, profile, or required artifact may fail closed before a report is returned. The CLI exits 0 for a passing process report and 2 for a returned failing report. A Completed run may still pass process integrity, but ReadyForHumanReleaseReview is true only while a passing run is AwaitingReleaseApproval.

7. Fresh-run repair: bounded feedback without approval drift

Revision is deliberately not an in-place patch. Hyperion verifies the actual parent run and then creates a new specification containing the original sources, root objective, exact parent Synthesizer output, exact parent Risk Judge review, required user-approved change request, and optional diagnostics supplied by the user or Terminal.

Diagnostics are accepted as supplied, hash-bound text. The free workflow neither creates nor executes them and does not prove that their claims are true.

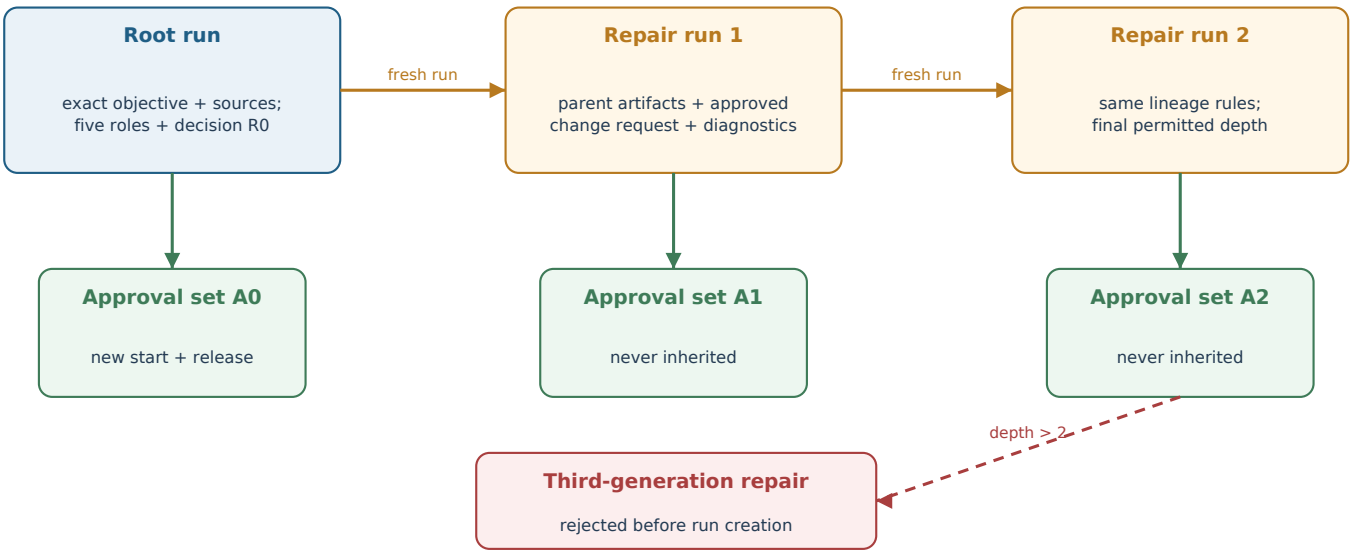


Figure 6. A lineage permits at most two repair generations. Each generation reruns all five roles and receives new approvals.

$$Lineage_r = (r, ParentRunId, h(ParentSpec), h(ParentDraft), h(ParentReview), h(ChangeRequest), h(Diagnostics))$$

Invariant	Enforcement
Verified ancestry	Every loaded repair recursively checks the actual parent event chain, status, provider, artifacts, decision, root objective, and round depth.
Provider continuity	Provider, model, protocol, endpoint, prices, and credential binding must match the parent.
Replacement, not patch	The new objective asks for one complete strategy; the previous bytes remain immutable evidence.
Fresh authority	No start or release approval is inherited. Every approved replacement binds new exact hashes.
Bounded depth	Round 1 and round 2 are permitted; round 3 is rejected before run creation.
Context headroom	Initial runs accept at most 58 user sources, reserving six of 64 slots for lineage records.
Preflight failure	Oversized inherited evidence or repair input fails before partial lineage is created.

CURRENT LIMIT

The store does not enforce one child per parent, so sibling repair runs may be created. The depth cap is per lineage, and each new run has its own configured budget; there is no implemented aggregate root-family cost cap.

8. Budget model, retries, and stop conditions

Bounds are part of the security and product contract. Before each provider call, Hyperion reserves capacity against request, token, byte, elapsed-time, and declared-cost ceilings. Invalid role output may consume another attempt, but it cannot create an unbounded self-correction loop.

Resource	Default per run	Validation ceiling
Provider requests	10	1,000
Attempts per role	2	20
Prompt tokens	500,000	10,000,000
Output tokens	40,000	10,000,000
Output tokens per request	8,000	1,000,000
Declared cost	USD 5	USD 1,000,000
Response bytes	1,000,000	100,000,000
Artifact bytes	2,000,000	100,000,000
Elapsed time	900 seconds	86,400 seconds
Request timeout	120 seconds	3,600 seconds
Repair generations	2	2, fixed

$$ChargedPromptTokens = max(ProviderReportedTokens, ConservativeUtf8Estimate)$$

Terminal outcomes

Condition	Outcome
Role exhausts its attempt count	Failed
Non-retryable provider error, truncation, or invalid terminal response	Failed
Request, token, byte, artifact, time, or declared-cost limit reached	BudgetExhausted
Operator cancellation token interrupts the invocation	Cancelled
Risk Judge says Revise	NeedsRevision
Risk Judge says Reject	Rejected
Risk Judge says Approve	AwaitingReleaseApproval
Exact human release approval succeeds	Completed

There is no current diagnostic-fingerprint or no-improvement detector. Fixed per-role attempts and two repair generations are the implemented outer bounds.

9. Bring-your-own-key provider routing

The CLI chooses a named profile instead of allowing every network field to vary. Each built-in remote profile fixes its origin, protocol, authentication adapter, and environment-variable slot. Secrets are resolved only when a live start needs them and are excluded from config, run specification, workspace, artifacts, and CLI output.

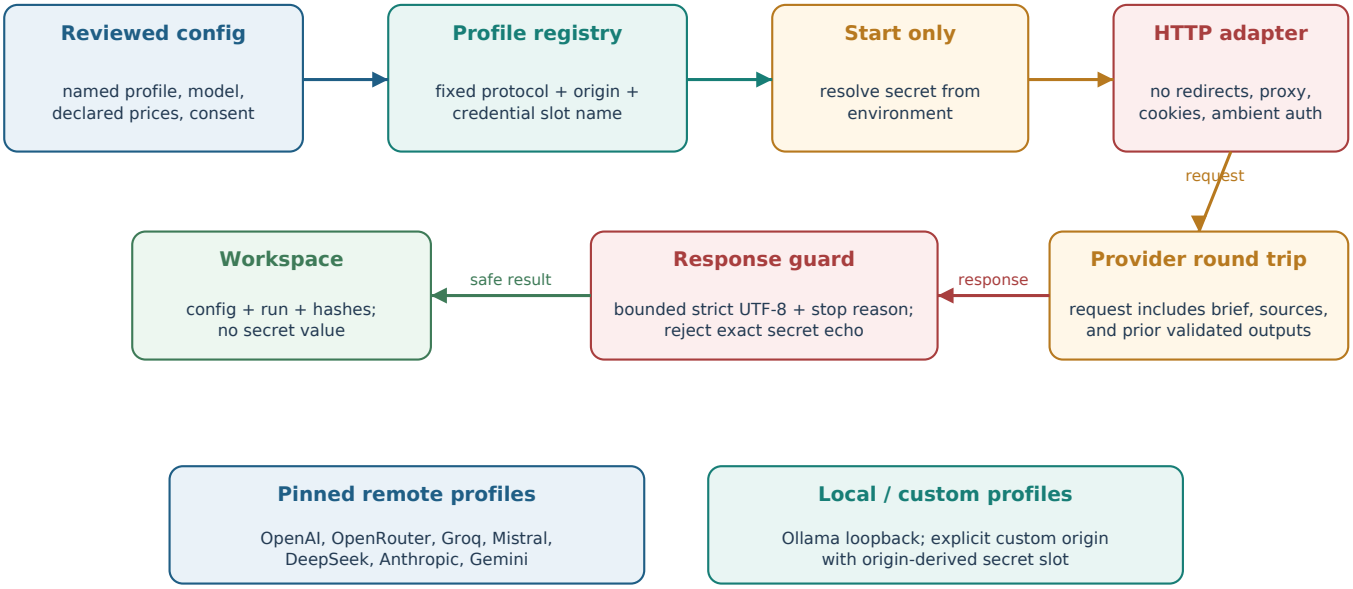


Figure 7. Credential routing and persistence boundary. The nonsecret slot name is hash-bound; the secret value is not stored.

Profile family	Protocol and boundary
mock	Network-free rehearsal with no provider key.
ollama	Loopback OpenAI-compatible HTTP at the pinned local origin; no key by default.
openai / openrouter / groq / mistral / deepseek	Pinned HTTPS origin and OpenAI-compatible adapter with a dedicated DAXALGO_LLM_* slot.
anthropic	Pinned official HTTPS origin with native Anthropic Messages protocol and authentication.
gemini	Pinned official HTTPS origin with native Gemini generateContent protocol and authentication.
custom-openai-compatible	Explicit custom-provider consent; canonical origin determines a full SHA-256 credential-slot suffix.
external-adapter/v1	Embedding Terminal owns endpoint, authentication, egress consent, credential store, and redaction.

OPENAI ROUTE LIMIT

The current OpenAI profile uses a constrained Chat Completions adapter and rejects o-series model IDs because its bounded request shape is incompatible with those models. Native OpenAI Responses support is deferred.

The routing controls do not declare a selected provider trustworthy. Hyperion grants the model no tools, but it cannot prove that an independently operated remote service or local daemon performs no activity outside the request.

10. Threat model and failure containment

The workflow assumes model output and supplied sources are untrusted. It limits what can leave the process, what can enter durable state, and what authority an accepted artifact receives. The table separates implemented mitigations from residual risk.

Threat	Implemented containment	Residual risk
Prompt injection in a source	Escape source records; label them untrusted data; fixed system policy; no model tools.	A model may still follow malicious prose or produce poor advice; downstream validation remains necessary.
Invented SDK or API	Evidence Analyst and Critic prompts; known source citations; Risk Judge review.	No compiler currently runs, so invented symbols may remain in inert source.
Artifact tampering	Content-addressed bytes, event-chain validation, exact artifact reconstruction, exact-hash approval.	Hashes identify changed bytes but cannot establish semantic quality.
Approval replay or drift	Exactly one approval at each gate; start binds spec; release binds spec and artifact; repairs inherit no approval.	Actor identity is local workflow evidence, not a public-key signature claim.
Credential exfiltration	Fixed origins and slots, no redirects/proxy/cookies/ambient auth, whitespace/control rejection, exact-secret echo filter.	A trusted provider receives the key by design; custom/private network and DNS behavior still require production hardening.
Provider endpoint drift	Pinned profile descriptor is part of the approved run binding; resume and repair reject mismatch.	Provider behavior behind the same origin may change without a capability fingerprint.
Runaway inference cost	Requests, attempts, tokens, bytes, time, and declared cost are bounded per run.	No aggregate budget across sibling repairs or an entire root family.
Crash after dispatch	Durable state, stale detection, conservative reservation charge, bounded retry.	External execution or billing may be duplicated without provider idempotency evidence.
Unsafe generated behavior	Artifact is inert, human release and Terminal compile consent required, installEligible false.	Safety, behavior, and profitability remain unverified until separate Terminal gates run.

Plaintext local workspace

The workspace contains the brief, supplied sources, role outputs, approvals, and generated source in plaintext SQLite and content-addressed files. Review commands may emit specifications or artifacts to standard output. Users must choose a private workspace, protect logs, and apply an appropriate retention policy. A production Terminal integration remains responsible for app-owned storage, operating-system-backed secrets, run-bound egress receipts, and safe deletion.

11. Operator walkthrough and evidence produced

The normal path has two explicit approvals and no hidden deployment step. The operator first approves the exact run specification, then reviews the generated artifact and process report before releasing those exact inert bytes to the next stage.

```
1. Configure: hyperion workflow init --config workflow.json --profile anthropic --model MODEL --input-price
PRICE --output-price PRICE --allow-provider-egress
2. Create: hyperion workflow create --config workflow.json --workspace PATH --brief brief.md --source
sdk=SDK.md
3. Approve and run: hyperion workflow start --config workflow.json --workspace PATH --run RUN_ID --spec
SHA256 --actor USER
4. Inspect: hyperion workflow gauntlet --config workflow.json --workspace PATH --run RUN_ID
5. Review: hyperion workflow show --config workflow.json --workspace PATH --run RUN_ID
6a. Release inert bytes: hyperion workflow release --config workflow.json --workspace PATH --run RUN_ID
--artifact SHA256 --actor USER
6b. Or repair: hyperion workflow revise --config workflow.json --workspace PATH --run RUN_ID --request
change.md --diagnostics diagnostics.json --owner USER
```

Evidence ledger

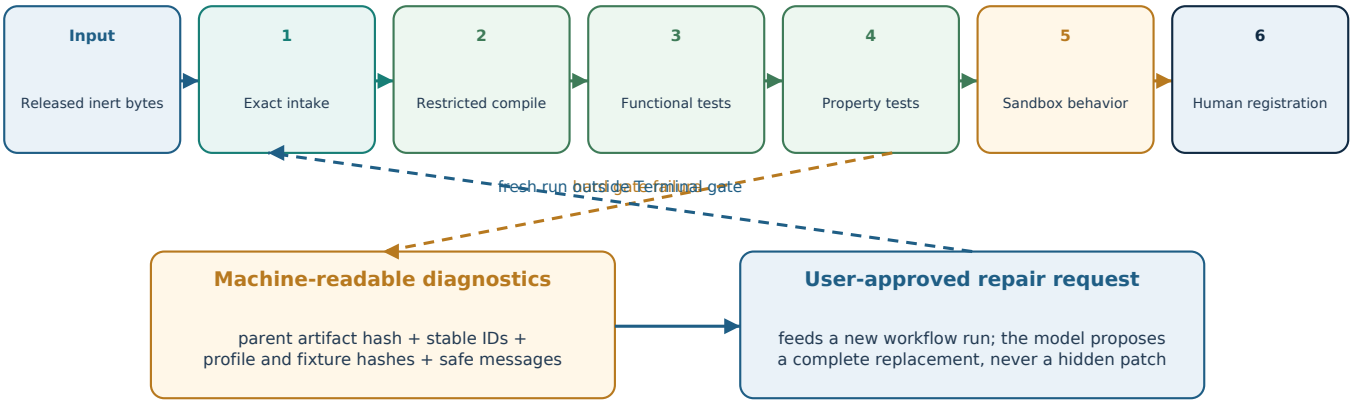
Moment	Human sees	Durable evidence
After create	Objective, sources, provider descriptor, budget, and specification SHA-256.	AwaitingStartApproval snapshot.
After start	Role progress, status, resource usage, and terminal decision.	Exactly one start approval plus hash-chained role events.
After gauntlet	Eight pass/fail checks and explicit not-run semantic fields.	Versioned gauntlet report derived from durable state.
Before release	Exact source files, evidence IDs, role outputs, risk decision, and artifact SHA-256.	AwaitingReleaseApproval with no release approval yet.
After release	Completed state for the same exact artifact.	Exactly one release approval bound to spec and artifact.
After revise	New run ID, new specification hash, explicit parent lineage, and new approval requirement.	Fresh AwaitingStartApproval run; parent remains immutable.

REVIEW QUESTION

The correct release question is: 'May these exact inert bytes proceed to the Terminal verification stage?' It is not: 'Is this strategy safe, profitable, compiled, or ready to trade?'

12. Proposed Terminal-owned verification gauntlet

After human release or import, the consuming Terminal should independently validate the exact artifact in a restricted environment. These gates are a design target, not behavior of the current workflow gauntlet.



This downstream compiler/test/property/sandbox gauntlet is PROPOSED and NOT implemented by workflow gauntlet

Figure 8. Proposed downstream verification and diagnostics feedback. Every repair returns to a fresh Hyperion run and then repeats all applicable Terminal gates from clean state.

Deferred gate	Required evidence	Failure disposition
1. Exact intake	Artifact, file, compiler profile, SDK/context pack, and source-manifest hashes.	Reject any drift.
2. Restricted compile	Pinned compiler/toolchain; no package restore, network, broker, or model authority; normalized diagnostics.	Hard fail; diagnostics may seed an approved repair.
3. Functional tests	Versioned golden and request-derived tests for lifecycle, state, signals, order intent, sizing, exits, and controls.	Hard fail; a model cannot waive an assertion.
4. Property tests	Determinism, bounds, causality, missing/extreme inputs, risk invariants, and declared metamorphic relations.	Hard fail or explicitly inconclusive; never semantic pass on inconclusive evidence.
5. Sandbox behavior	Frozen fixtures, bounded CPU/memory/time, no network or broker, exception and trace records.	Terminate on authority or resource violation.
6. Human registration	Source, diagnostics, tests, properties, residual risks, and exact hashes.	No registration without separate Terminal authority.

Synthetic fixtures produced for these gates would be workflow-verification evidence only. They must not be described as model-training evidence or as proof of trading quality.

13. Assurance boundary and evaluation protocol

A reliable review separates process integrity from program behavior. The implemented gauntlet can establish provenance, completeness, structural validity, and approval consistency for exact bytes. It cannot establish claims that require parsing, compilation, execution, market data, or statistical evaluation.

A passing current gauntlet establishes	It does not establish
Exact policy, workflow, prompt catalog, provider, budget, specification, and event lineage.	That the provider is truthful, independent, stable, or high quality.
Complete structurally valid role artifacts and an approving Risk Judge output.	That the Risk Judge is a formal verifier or that agent debate occurred.
Reproduction of the inert strategy artifact from exact approved lineage.	Parsing, compilation, linking, loading, execution, or behavioral correctness.
Bounded workflow authority and no install eligibility.	Absence of unsafe logic, lookahead, leakage, numerical defects, or excessive risk.
Human approvals bound to exact hashes.	Profitability, robustness, suitability, compliance, or live-trading safety.

Recommended evaluation layers

Layer	Metric	Interpretation
Workflow integrity	Eight-check pass rate; corrupted-state rejection; approval-binding tests.	Does the coordinator fail closed and preserve exact lineage?
Generation utility	Structural-validity rate, citation validity, provider cost, latency, repair rate.	Can the selected model satisfy the narrow output contract efficiently?
Compiler evidence	Compile-at-one and compile-after-repair under a pinned toolchain.	Do exact artifacts satisfy syntax, types, and SDK contracts?
Behavioral evidence	Functional, property, metamorphic, and sandbox pass rates on sealed fixtures.	Do tested behaviors match declared requirements and invariants?
Trading research	Out-of-sample protocol, leakage controls, robustness, uncertainty, and economic assumptions.	Separate research evidence; never implied by code-generation success.

Acceptance rule

$$Proceed = ProcessIntegrity \text{ AND } HumanRelease \text{ AND } TerminalHardGates \text{ AND } RegistrationConsent$$

Each term is necessary; none may be replaced by model confidence or persuasive prose. Even when all proposed Terminal gates pass, the evidence is scoped to the exact artifact, toolchain, profiles, fixtures, and tests used. It is not a universal proof of profitability or behavior under all market conditions.

14. Implementation map and reproducibility

The paper is grounded in the repository artifacts below. Normative version strings and limits should be read from code and the manifest if this explanatory paper ever becomes stale.

Concern	Repository anchor
Workflow contracts, defaults, gauntlet report, repair lineage	src/windows/AI/Hyperion.FreeStrategyWorkflow/FreeStrategyWorkflowContracts.cs
Create, start, gauntlet, release, recursive repair validation	src/windows/AI/Hyperion.FreeStrategyWorkflow/FreeStrategyWorkflowRunner.cs
Five role instructions and prompt-catalog hash	src/windows/AI/TradingTerminal.Ai.Coordinator/Orchestration/CoordinatorPromptCatalog.cs
Coordinator state machine	src/windows/AI/TradingTerminal.Ai.Coordinator/Orchestration/ResearchCoordinator.cs
CLI commands and disclosures	src/windows/Tools/Hyperion.Cli/FreeStrategyWorkflowCli.cs
Pinned provider profiles	src/windows/Tools/Hyperion.Cli/FreeStrategyWorkflowCliConfig.cs
Provider runtime and secret resolution	src/windows/Tools/Hyperion.Cli/FreeStrategyWorkflowCliRuntime.cs
Native Anthropic and Gemini adapters	src/windows/AI/TradingTerminal.Ai.Coordinator/Providers/
Distributable contract	products/free-strategy-workflow/manifests/workflow.v1.json
Research comparison and future design	products/free-strategy-workflow/planning/agentic-gauntlet-design.md
Workflow and provider tests	tests/TradingTerminal.Tests.Pro.Headless/Coordinator/

Version snapshot

Field	Value
Workflow schema	hyperion-free-strategy-workflow/v1
Gauntlet schema	hyperion-free-strategy-gauntlet-report/v1
Repair schema	hyperion-free-strategy-repair-lineage/v1
Strategy artifact schema	coordinator-strategy-source/v1
Maximum repair rounds	2
Maximum initial user sources	58
Model tools	none
SemanticCorrectnessGuaranteed	false
CompilationVerification	not-run-by-free-workflow
FunctionalVerification	not-run-by-free-workflow

REPRODUCIBILITY NOTE

This PDF is generated from a repository-owned Python builder using ReportLab. It embeds fonts, uses vector diagrams and tables, reads the current workflow manifest, and is visually verified by rendering every page through Poppler.

References

References describe design precedents and limitations. Their reported results are not Hyperion benchmark results, and citation does not imply that Hyperion implements the cited system in full.

[1] Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. https://arxiv.org/abs/2210.03629

[2] Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. https://arxiv.org/abs/2303.11366

[3] Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. https://arxiv.org/abs/2303.17651

[4] Zhou et al. Language Agent Tree Search Unifies Reasoning, Acting, and Planning. https://arxiv.org/abs/2310.04406

[5] Du et al. Improving Factuality and Reasoning through Multiagent Debate. https://arxiv.org/abs/2305.14325

[6] Gou et al. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. https://arxiv.org/abs/2305.11738

[7] Chen et al. Teaching Large Language Models to Self-Debug. https://arxiv.org/abs/2304.05128

[8] Ridnik et al. Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering. https://arxiv.org/abs/2401.08500

[9] Chen et al. CodeT: Code Generation with Generated Tests. https://arxiv.org/abs/2207.10397

[10] Liu et al. Is Your Code Generated by ChatGPT Really Correct? EvalPlus. https://arxiv.org/abs/2305.01210

[11] Huang et al. Large Language Models Cannot Self-Correct Reasoning Yet. https://arxiv.org/abs/2310.01798

[12] OpenAI. Introducing deep research. https://openai.com/index/introducing-deep-research/

[13] Anthropic. How we built our multi-agent research system. https://www.anthropic.com/engineering/multi-agent-research-system

[14] OpenAI API reference. Create chat completion. https://developers.openai.com/api/reference/resources/chat/subresources/completions/methods/create

Conclusion

Hyperion's free CLI workflow is best understood as a bounded, durable authoring protocol around an untrusted model. Five fixed roles improve decomposition and review coverage; exact artifacts, hash-bound approvals, process gates, and fresh-run repair preserve accountability. The model proposes. Hyperion records and constrains. Humans release exact inert bytes. The Terminal must separately compile, test, sandbox, and authorize any later use.

ONE-SENTENCE SYSTEM MODEL

A serial role team produces a reviewable proposal; deterministic infrastructure decides whether the proposal's process evidence is intact; separate Terminal authority decides whether exact bytes may ever become executable.